INTEGRATION GUIDE — **Full, Updated** (Producers, Consumer, Spark Ops, Streamlit, Build)

This guide explains **exactly** how each teammate integrates their part into the project and runs the full pipeline.
It reflects the **current repo structure** and the changes we made today (Spark Ops registry, two `team_config.yaml` files, `.mk` helpers, build targets, etc.).

---

0) What you add as a teammate

You contribute three things (minimum is just #1):

1. **Producer** — `src/producers/<prefix>_producer.py` (pulls your Spotify data and publishes JSON events to Kafka).
2. **(optional) Spark operator** — `src/spark/ops/<prefix>_op.py` + register it in `registry.py` + configure it in `src/spark/team_config.yaml`.
3. **(optional) Streamlit tab** — `src/app/tabs/<prefix>.py` + register it under your prefix in `src/app/team_config.yaml`.

`<prefix>` is your short ID (e.g., `alex`, `gilian`, `vadim`). **Use the same prefix everywhere** (env, Kafka topics, Mongo collections, Spark config, UI).

---

1) Project architecture

```
Producer → Kafka → Consumer → MongoDB → Spark (ops/registry) → Streamlit UI
```

- **Kafka** streams your JSON events.
- **Consumer** writes each topic to a **Mongo collection with the same name** (or a mapped one via `KAFKA_TOPIC_ROUTES`).
- **Spark** aggregates/analyses your data **in micro-batches** using a **mode** from `ops/registry.py`.
- **Streamlit** visualizes results from Mongo, per teammate tab.
- Everything runs with **Docker Compose**. No local venv needed.

---

2) Repository layout (current)

```
src/
├── app/                        # Streamlit
│   ├── streamlit_app.py
│   ├── tabs/
│   │   ├── avd.py / alex.py / gilian.py / vadim.py / raw.py /
spark_generic.py ...
│   └── team_config.yaml        # UI tabs per teammate
```

```
├── consumers/                # Kafka → Mongo writer
│   └── kafka_consumer.py
├── docker/                   # Dockerfiles per service
├── docker-compose.yml
├── envs/                     # .env.<prefix> lives here (you create yours)
├── lib/                      # shared utilities (Kafka producer helpers,
payload normalizers)
├── make/                     # optional personal .mk shortcuts (see §10)
│   ├── avd.mk
│   └── _template.mk
├── makefile                  # main Make targets (infra, build, run)
├── producers/                # your <prefix>_producer.py lives here
│   ├── avd_producer.py
│   └── example_producer.py
├── spark/                    # Spark Structured Streaming
│   ├── streaming_job.py
│   ├── team_config.yaml      # Spark config per teammate (mode + params)
│   ├── ops/                  # NEW modular operators (Option B)
│   │   ├── base.py
│   │   ├── top_artist.py
│   │   ├── top_tracks.py
│   │   ├── feature_avg.py
│   │   ├── my_custom_op_template.py
│   │   ├── registry.py
│   │   └── __init__.py
│   ├── schemas.py
│   └── requirements.txt
└── tools/
    └── install_docker_ubuntu.sh   # optional: installs Docker on Ubuntu
```

> **Two config files named `team_config.yaml` exist — by design:**
> • `src/spark/team_config.yaml` → **Spark**: chooses operator *mode* and parameters per teammate.
> • `src/app/team_config.yaml` → **Streamlit**: declares tabs per teammate (UI only).

---

3) Environment: create your `.env.<prefix>`

Create `src/envs/.env.<prefix>` by copying Dorin's example and editing your values:

```
# Identity
APP_PREFIX=<prefix>              # e.g., alex

# Spotify API (YOUR credentials)
CLIENT_ID=...
CLIENT_SECRET=...
USERNAME=...
REDIRECT_URI=http://127.0.0.1:8888/callback

# Optional per-user market for Top10
MARKET_OVERRIDE=DE
```

```
# Flags
DEBUG=true
KAFKA_ENABLED=true

# Kafka (inside Docker network)
KAFKA_BOOTSTRAP=kafka:19092

# Your topics (comma-separated). Convention:
#   <prefix>_recent_events
#   <prefix>_artist_market_top_tracks
KAFKA_TOPICS=${APP_PREFIX}_recent_events,${APP_PREFIX}_artist_market_top_tr
acks

# Optional routing: topic → semantic family (consumer uses it to shape
docs)
KAFKA_TOPIC_ROUTES=${APP_PREFIX}_recent_events:events,${APP_PREFIX}_artist_
market_top_tracks:top10

# Mongo (inside Docker network)
MONGO_URL=mongodb://root:example@mongo:27017/?authSource=admin
MONGO_DB=spotify_db

# Default collections used in app/consumer
MONGO_COLL_EVENTS=${APP_PREFIX}_recent_events
MONGO_COLL_TOP10=${APP_PREFIX}_artist_market_top_tracks

# Producer entrypoint (module name in src/producers/ without .py)
PRODUCER_ENTRY=${APP_PREFIX}_producer

# Spark defaults (used only if Spark team_config.yaml misses your prefix)
SPARK_MODE=top_artists
SPARK_FEATURE=track_duration_ms
SPARK_GROUP=market_used
SPARK_TOPN=10

# Needed Spark connector jars
SPARK_PACKAGES=org.apache.spark:spark-sql-kafka-0-
10_2.12:3.5.1,org.apache.kafka:kafka-clients:3.5.2

# Streamlit port (host mapping)
PORT=8501
```

Load your env into `.env`:

```
# Option A: if the Makefile has a dedicated shortcut for you (e.g. alex-
env)
make <prefix>-env

# Option B: universal loader (works for any prefix)
make env-user USR=<prefix>
make show-env
```

> **Never commit** your real `.env.<prefix>` (it's in `.gitignore`). Commit an example named `.env.<prefix>.example` if needed.

---

4) Build — IMPORTANT (first time and whenever deps change)

Build everything (producer, consumer, spark, app):

```
# If the Makefile already has these convenience targets (recommended)
make build          # builds all images
# or, if you need a clean rebuild after changing Dockerfiles/requirements:
make rebuild        # no-cache, pull fresh bases

# If those targets do NOT exist in your local Makefile yet, use per-
service:
make producer-build
make consumer-build
make spark-build
make app-build
```

Clean/reset when things get weird (ports, stale volumes, etc.):

```
# clean all from Docker, DBs, all!!!!!!!!!!!!!!!!:
make clean

# Otherwise:
docker compose down -v    # removes containers + volumes
```

---

5) Start the stack and wire topics

```
make up                         # start infra (Kafka, ZK, Mongo, UI, etc.
as defined)
make kafka-init-from-env        # create topics from your .env (idempotent)
```

Run the services:

```
make consumer-run               # Kafka → Mongo writer (shared)
make producer-run               # your producer (uses PRODUCER_ENTRY)
make spark-up                   # Spark streaming job (runs forever)
make app-up                     # Streamlit UI
```

Open:

- **Streamlit** → http://localhost:8501
- **Mongo Express** → http://localhost:8081

---

6) Producer — where to put your code

1. Copy the example and adapt it:

```
cp src/producers/example_producer.py src/producers/<prefix>_producer.py
```

2. In your producer file, **emit one JSON per event** to your topic(s) from `.env`. Use the helpers in `src/lib/` if useful.
3. Ensure `.env.<prefix>` sets `PRODUCER_ENTRY=<prefix>_producer` and `KAFKA_TOPICS=...`.
4. Rebuild or re-run as needed (see §4 and §5).

> **Topic → collection:** the consumer writes each topic to a **Mongo collection with the same name** by default. You can map topics via `KAFKA_TOPIC_ROUTES` if you need a different collection name or special handling (events vs top10).

---

7) Spark — how your mode is chosen

Spark loads **`src/spark/team_config.yaml`** and picks the section under your `APP_PREFIX`. Example:

```yaml
teams:
  avd:
    mode: top_artists
    topn: 10

  alex:
    mode: feature_avg
    feature: energy
    group: market_used
    topn: 10

  gilian:
    mode: top_tracks

  vadim:
    mode: my_custom_mode
    feature: danceability
    group: market_used
    topn: 15
```

- The **mode** must exist in `src/spark/ops/registry.py`.

- If your prefix is missing from this YAML, Spark falls back to the **env vars** `SPARK_MODE/SPARK_FEATURE/SPARK_GROUP/SPARK_TOPN` from `.env`.

The output collection

Spark writes its results to Mongo (via `foreachBatch`) into a collection named:

```
<APP_PREFIX>_spark_<mode>
```

Examples: `alex_spark_feature_avg`, `avd_spark_top_artists`, `vadim_spark_my_custom_mode`.

---

8) Spark Ops (modular)

Operators live in `src/spark/ops/`. Each op is a **callable**:
`(df: DataFrame, cfg: dict|None) -> DataFrame`

Available defaults:

- `top_artists` → Top-N plays by `artist_names`
- `top_tracks` → Top-N plays by `track_name`
- `feature_avg` → Average of a numeric field per group(s)

Create your own op in 4 steps:

1. Copy the template:

```
cp src/spark/ops/my_custom_op_template.py src/spark/ops/<prefix>_op.py
```

2. Change the function name inside (e.g. `build_avg_valence_by_market`) and implement your logic.
3. Register it in the registry:

```
# src/spark/ops/registry.py
from .<prefix>_op import build_my_custom_op   # rename to your function
OPS["my_custom_mode"] = build_my_custom_op
```

4. Configure it under your prefix in `src/spark/team_config.yaml`:

```
teams:
  <prefix>:
    mode: my_custom_mode
    feature: danceability
    group: market_used
    topn: 15
```

The input `df` is already **parsed & normalized** (array-like artist fields normalized, all optional fields added if missing). **Keep outputs small** (Top-N) — the console sink and UI stay readable.

---

9) Streamlit — add your tab

Create `src/app/tabs/<prefix>.py` with a function such as:

```
def render(db, cfg, prefix):
    st.header(f"{prefix.upper()} Dashboard")
    # Query your collections; use cfg + prefix for names:
    #   events: f"{prefix}_recent_events"
    #   top10 : f"{prefix}_artist_market_top_tracks"
    #   spark : f"{prefix}_spark_<mode>"
    # Then build charts/tables for your story.
```

Register your tab in `src/app/team_config.yaml`:

```
team:
  - prefix: <prefix>
    display_name: Your Name
    tabs:
      - <prefix>
```

---

10) `.mk` helper files (recommended)

Folder `src/make/` can contain **personal Makefile snippets**. The repo already includes:

- `src/make/avd.mk` — Dorin's shortcuts
- `src/make/_template.mk` — a template you can copy

How to use: create `src/make/<prefix>.mk` and define your shortcuts, for example:

```
# src/make/<prefix>.mk
.PHONY: <prefix>-env <prefix>-demo

<prefix>-env:
    @cp envs/.env.<prefix> .env
    @echo "[ENV] Loaded .env.<prefix> for <prefix>"

<prefix>-demo: <prefix>-env
    docker compose up -d zookeeper kafka mongo
    make kafka-init-from-env
    docker compose run --rm -e PRODUCER_ENTRY=<prefix>_producer -e
PYTHONPATH=/app/src producer
    docker compose up -d spark-app app
    @echo "[INFO] Demo up and running for <prefix>!"
```

> The main `makefile` already defines general targets (`env-user`, `up`, `producer-run`, `spark-up`, `app-up`, etc.). Your `<prefix>.mk` just adds **friendly aliases** so you type less.
> If your main `makefile` supports `include` of `src/make/*.mk`, you're done. If not, you can run the universal target: `make env-user USR=<prefix>`.

---

11) Build / Run command table (for quick copy-paste)

Task	Command(s)
Load your env	`make <prefix>-env` **or** `make env-user USR=<prefix>`
Build all (if available)	`make build`
Rebuild no-cache	`make rebuild`
Build per service	`make producer-build && make consumer-build && make spark-build && make app-build`
Start infra	`make up`
Create topics	`make kafka-init-from-env`
Run consumer	`make consumer-run`
Run your producer	`make producer-run`
Run Spark	`make spark-up`
Run Streamlit	`make app-up`
Tail logs	`make <service>-logs` (producer/app/spark)
Stop everything	`make down`
Clean (if available)	`make clean` or `docker compose down -v`

---

12) Offsets & “where is my data?”

Kafka consumers **don’t read the past** by default if they start late.

- **Best practice:** start **consumer first**, then run the **producer again**:

```
make consumer-run
make producer-run
```

- **Reset offsets** (advanced):

```
docker compose exec kafka kafka-consumer-groups     --bootstrap-server
kafka:19092     --group <your_consumer_group>     --reset-offsets --
all-topics --to-earliest --execute
```

---

13) Access URLs

- Streamlit: **http://localhost:8501**
- Mongo Express: **http://localhost:8081**

---

14) What to commit (and what not)

Commit:

- `src/producers/<prefix>_producer.py`
- `src/spark/ops/<prefix>_op.py` + `registry.py` edit
- `src/app/tabs/<prefix>.py`
- `src/spark/team_config.yaml` (your section)
- `src/app/team_config.yaml` (your tab)
- `.env.<prefix>.example` (if you want to share variable names without secrets)

Do NOT commit:

- Real `.env` files (with secrets)
- Spotify tokens/caches
- Random OS/editor caches

---

15) Docker install (Ubuntu one-liner)

If you're on Ubuntu and don't have Docker yet, run the helper script:

```
bash src/tools/install_docker_ubuntu.sh
```

Then **log out/in** or run `newgrp docker` so `docker` works without `sudo`.

---

16) Troubleshooting checklist

- **No documents in Mongo for `<prefix>_recent_events`:**
  Start consumer, then rerun producer; or reset offsets.

- **Spark prints warning "TEAM_CONFIG is empty":**
  Make sure `src/spark/team_config.yaml` exists **in the container context** (you committed it to

repo) and contains your prefix. If missing, Spark falls back to `SPARK_MODE` in `.env`.

- **Port already in use (8081/8501/9092):**
  `make down` → kill stray `docker-proxy` on that port → `make up`.
- **Streamlit tab missing:**
  Add your file in `src/app/tabs/<prefix>.py` and register in `src/app/team_config.yaml`.
- **Mode not found:**
  Ensure your op name is registered in `src/spark/ops/registry.py`, and the same name is used under your prefix in `src/spark/team_config.yaml`.

---

17) Final sequence

```
# 1) Create src/envs/.env.<prefix> (copy Dorin's, edit secrets)
make env-user USR=<prefix>     # or make <prefix>-env

# 2) Build images
make build                      # or per-service builds

# 3) Start infra and topics
make up
make kafka-init-from-env

# 4) Start processing pipeline
make consumer-run               # 4a: writer to Mongo
make producer-run               # 4b: publish your events (rerun any time)
make spark-up                   # 4c: aggregations to <prefix>_spark_<mode>
make app-up                     # 4d: open http://localhost:8501
```

That's it. If something doesn't work, re-check your prefix/name consistency and follow the Troubleshooting section.